Professor (PID, pname, dept)

Course (code, dept Id, c name, syllabus)

Prof_course (PID, code, semester)

Q) Identify PK and FK for schema and populate this relation with some tuples and give some examples of insertion operation in the Professor_post course rel^n that violates the referential integrity constraints, and of another that doesn't. Identify various candidate keys for this schema. Make necessary assumptions wherever necessary.

CUSTOMER (custno, cust_name, address)

ORDER (orderno, orderdate, custno, city, amount)

PRODUCT (prodno, price, order no)

Q) Specify FK constraints for above DB. Also insert some tuples in the relation. Show some e.g. of deletion of tuples that violates referential integrity constraints. Make any assumptions wherever necessary.

Relational Algebra

- Formal language for representing relational model

(i) Unary operator — σ, π

(ii) Binary operator — $\cup, \cap, X, -, \div, \bowtie$

σ (Select Operator)

Syntax: $\sigma_{\text{<condition>}}(\text{<tablename>})$

e.g.- emp (ename, id, sal, dno, address)

$\sigma_{\text{address = "Patna"}}(\text{emp})$

↑ equal to

$\sigma_{\text{sal <> 10000}}(\text{emp})$

↑ not equal to

$\sigma_{\text{address = "Patna"} \wedge \text{sal} \geq 10000}(\text{emp})$

↑ and

$\sigma_{\text{address = "Patna"} \vee \text{"Delhi"}}(\text{emp})$

↑ or

* for retrieval of tuples

π (Projection)

Syntax:

π <table name>
<column name, column name 2, ... >

* for retrieval of columns

e.g.- $\pi_{ename, sal}(emp)$ ← to select name and sal from emp

Q) Select name and sal of all employees that live in Patna having salary greater than 10000

$R_1 \leftarrow \sigma_{address = "PATNA" \wedge sal > 10000}(emp)$

R_1:

ename	id	sal	dno	address
abc	--	12000	--	PATNA
def	--	20000	--	PATNA

$Result \leftarrow \pi_{ename, sal}(R_1)$

Result:

ename	sal
abc	12000
def	20000

OR

$Result \leftarrow \pi_{ename, sal}(\sigma_{address = "PATNA" \wedge sal > 10000}(emp))$

$\cup$ (Union operator)

- Degree of both relations (no. of attributes) is same

R_1:

id	name
01	aaa
02	bbb
03	ccc

R_2:

id	name
04	ddd
05	eee
06	fff
01	aaa

$R_1 \cup R_2$:

id	name
01	aaa
02	bbb
03	ccc
04	ddd
05	eee
06	fff

$R_1 \cap R_2$:

id	name
01	aaa

− (Difference Operator)

$R_1 - R_2$:

id	name
02	bbb
03	ccc

$R_2 - R_1$:

id	name
04	ddd
05	eee
06	fff

X (Cartesian Product)

emp

eno	ename	sal	dno
2A	aaa	1000	1A
2B	bbb	2000	1A
2C	ccc	3000	1B
2D	ddd	4000	1C

dept

dno	dname
1A	BTech
1B	BCA
1C	BBA

emp X dept:

eno	ename	sal	dno	~~dname~~ dno	dname
2A	aaa	1000	1A	1A	BTech
2A	aaa	1000	1A	1B	BCA
2A	aaa	1000	1A	1C	BBA
2B	bbb	2000	1A	1A	BTech
2B	bbb	2000	1A	1B	BCA
2B	bbb	2000	1A	1C	BBA
2C	ccc	3000	1B	1A	BTech
2C	ccc	3000	1B	1B	BCA
2C	ccc	3000	1B	1C	BBA
2D	ddd	4000	1C	1A	BTech
2D	ddd	4000	1C	1B	BCA
2D	ddd	4000	1C	1C	BBA

emp ⋈ dept:

① "Natural Join" → only has the common attribute once

eno	ename	sal	dno	dname
2A	aaa	1000	1A	BTech
2B	bbb	2000	1A	BTech
2C	ccc	3000	1B	BCA
2D	ddd	4000	1C	BBA

② "Equi Join"
↳ joined without any condition, only based on common attribute

emp(dno) = dept(dno)

eno	ename	sal	dno	dno	dname
2A	aaa	1000	1A	1A	BTech
2B	bbb	2000	1A	1A	BTech
2C	ccc	3000	1B	1B	BCA
2D	ddd	4000	1C	1C	BBA

Natural Join
- ↳ Inner Join (⋈) → all matching tuples only
- ↳ Outer Join
  - → Left (⋈) → all matching tuples + left unmatched ones
  - → Right (⋈) → " " " + right " "
  - → Full (⋈) → " " " + both " "

÷ (Division Operator)

- "for all" objects having specific properties

$R_1 \div R_2$: set of all attributes 'A' of R_1 such that

R_1:

A	B
a_1	b_1
a_1	b_2
a_1	b_3
a_1	b_4
a_2	b_1
a_2	b_2

R_3:

B
b_2
b_4

$R_1 \div R_3$:

A
a_1
a_4

~~If (aggregation operator)~~

If (emp)

γ_{dname} max(sal), avg(sal), min(sal), sum(sal)

- "Grouping function" or "Aggregation function"

Q.) CUSTOMER (custno, cname, city)

ORDER (orderno, orderdate, custno, amount)

ORDER_ITEM (orderno, itemno, quantity)

ITEM (itemno, unitprice)

(i) Retrieve the order no and orderdate placed by the customer residing in "PATNA" city.

$R_1 \leftarrow$ (CUSTOMER $\bowtie$ ORDER)

$\bowtie$ CUSTOMER.custno = ORDER.custno

natural join

for common attributes (tablename.attributename)

$R_2 \leftarrow \sigma^{(R_1)}_{city="PATNA"}$

$Result \leftarrow \Pi^{(R_2)}_{orderno, orderdate}$

(OR)

$Result \leftarrow \Pi_{orderno, orderdata}\left(\sigma_{city="PATNA"}(CUSTOMER \bowtie ORDER)\right)$

CUSTOMER.custno = ORDER.custno

(ii) Retrieve the order no and unit price for item for which an order of Qty > 50.

$\Pi_{orderno, unitprice}\left(\sigma_{quantity>50}(ORDER_ITEM \bowtie ITEM)\right)$

ORDER_ITEM.itemno = ITEM.itemno

(iii) Retrieve the order no, date and item no for order of item having the price > 20

$\Pi_{orderno, orderdate, itemno}\left(\sigma_{unitprice>20}(((ORDER \bowtie ORDER_ITEM) \bowtie ITEM)\right)$

ORDER.orderno = ORDERITEM.orderno

ITEM.itemno = ORDER_ITEM.itemno

(iv) Retrieve the no. and unitprice of item for which an order is placed by customer C001

$$\Pi_{itemno,\ unitprice}\left(\sigma_{CUSTNO = C001}\left(\left(ORDER \bowtie_{ORDER.orderno = ORDER_ITEM.orderno} ORDER_ITEM\right) \bowtie_{ORDER_ITEM.itemno = ITEM.itemno} ITEM\right)\right)$$

Q.) Employee (empname, street, city)

Works (empname, compname, sal)

Company (compname, city)

Manages (empname, managername)

(i) Find the names of all employees who work for SBI.

$$\Pi_{empname}\left(\sigma_{compname = \text{"SBI"}}(Works)\right)$$

Employee ⋈ Company

Employee

(ii) Find name, street address and cities of residence of all employees who work for SBI and earn more than 20K p.a.

$$\Pi_{empname,\ street,\ city}\left(\sigma_{compname = \text{"SBI"} \wedge sal > 20000}\left(Employee \bowtie_{Employee.empname = Works.empname} Works\right)\right)$$

(iii) Find names of all employees in the DB who live in the same city as the company for which they work

$$\Pi_{empname}\left(\sigma_{Company.city = Employee.city}\left(\left(Works \bowtie_{Works.compname = Company.compname} Company\right) \bowtie_{Works.empname = Employee.empname} Employee\right)\right)$$

Employee

(iv) Find the no. of employees working in each company

$$\gamma_{\text{compname}}\, C_{\text{count(empname)}}\,(\text{Works})$$

(v) Find the avg, max^m and min^m salaries for each company

$$\gamma_{\text{compname}}\, C_{\text{avg(sal), max(sal), min(sal)}}\,(\text{Works})$$

Q) Author (AID , aname, state, city, zip, phone, URL)
Book (ISBN , booktitle, category, price , ~~copyright~~ date, year, page_count, pid)
Publisher (pid , pname, address, state, ~~compname~~, phone, emailid)
Author-Book (AID, ISBN)
Review (Rid, ISBN, Rating)

(i) Retrieve the city, phone, and URL of Author whose name is Ram

$$\Pi_{\text{city, phone, URL}}\left(\sigma^{(\text{Author})}_{\text{aname='Ram'}}\right) \;\Big|\; \{t.\text{city}, t.\text{phone}, t.\text{URL} \mid \text{Author}(t) \wedge t.\text{name} = \text{'Ram'}\}$$

(ii) Retrieve name, address and phone of all publishers, located in Bihar state

$$\Pi_{\text{pname, address, phone}}\left(\sigma^{(\text{Publisher})}_{\text{state='Bihar'}}\right) \;\Big|\; \{t.\text{name}, t.\text{address}, t.\text{phone} \mid \text{Publisher}(t) \wedge t.\text{state} = \text{'Bihar'}\}$$

(iii) Retrieve title and price of all textbooks with a pagecount > 600.

$$\Pi_{\text{booktitle, price}}\left(\sigma^{(\text{Book})}_{\text{page_count} > 600 \wedge \text{category='textbook'}}\right) \;\Big|\; \{t.\text{booktitle}, t.\text{price} \mid \text{Book}(t) \wedge t.\text{page_count} > 600 \wedge t.\text{category} = \text{'textbook'}\}$$

(iv) Retrieve the ISBN, title and price of the books belonging to either novel or language category.

$$\Pi_{\text{ISBN, booktitle, price}}\left(\sigma^{(\text{Book})}_{\text{category='Novel'} \vee_{\lambda} \text{'Language'}}\right) \;\Big|\; \{t.\text{ISBN}, t.\text{booktitle}, t.\text{price} \mid \text{Book}(t) \wedge (t.\text{category} = \text{'Novel'} \vee t.\text{category} = \text{'Language'})\}$$

(category=)

(V) Retrieve the id, name, address and phone ... novels

$$\Pi_{pid, pname, address, phone}\left(\sigma_{category='novel'}\left(Book \bowtie_{Publisher.pid=Book.pid} Publisher\right)\right)$$

$\{t.pid, t.pname, t.address, t.phone \mid Publisher(t) \wedge Book(s) \wedge s.pid = t.pid \wedge s.category = 'Novel'\}$

or $\wedge(\exists s)\, Book(s)$

(VI) Retrieve title and price of all the books published by "Hills Publications".

$$\Pi_{booktitle, price}\left(\sigma_{pname='Hills\ Publication'}\left(Book \bowtie_{Publisher.pid=Book.pid} Publisher\right)\right)$$

(VII) Retrieve title, reviewer's id and rating of all textbooks

$$\Pi_{booktitle, Rid, Rating}\left(\sigma_{category='textbook'}\left(Book \bowtie_{Book.ISBN=Review.ISBN} Review\right)\right)$$

$\{t.booktitle, s.Rid, s.Rating \mid Book(t) \wedge Rating(s) \wedge t.ISBN = s.ISBN \wedge t.category = 'textbook'\}$

(VIII) Retrieve id, name, URL of the authors and category of book C++

$$\Pi_{AuthorAID, aname, URL}\left(\sigma_{category='C++'}\left(\left(Book \bowtie_{Book.ISBN=Author\text{-}Book.ISBN} Author\text{-}Book\right) \bowtie_{Author.AID=Author\text{-}Book.AID} Author\right)\right)$$

(IX) Retrieve title, category and price of all the books written by 'Sham'.

$$\Pi_{booktitle, category, price}\left(\sigma_{aname='Sham'}\left(\left(Book \bowtie_{Book.ISBN=Author\text{-}Book.ISBN} Author\text{-}Book\right) \bowtie_{Author.AID=Author\text{-}Book.AID} Author\right)\right)$$

$\{t.booktitle, t.category, t.price \mid Book(t) \wedge (\exists s)(\exists p)(Author(s) \wedge Author_Book(p) \wedge t.ISBN = p.ISBN \wedge s.AID = p.AID$

(X) Retrieve name and address of publishers who have not published any book. $\wedge s.aname = 'Ram$

$$R \leftarrow \Pi_{pid}^{(Publishers)} - \Pi_{pid}^{(Books)}$$

$$result \leftarrow \Pi_{pname, address}\left(R \bowtie_{R.pid = Publishers.pid} Publishers\right)$$

(X) Retrieve the name of all publishers, ISBN, booktitle published by them.

$$\Pi_{\text{pname, ISBN, booktitle}} \left(\text{Publishers} \bowtie_{\text{Publishers.pid = Book.pid}} \text{Book} \right)$$

(ii) we can also do natural join

(XI) Retrieve id and no. of books written by each author

$$\gamma_{\text{AID, count(ISBN)}} (\text{Author-Book})$$

if we want aname, let the above be R,

$$\Pi_{\text{aname}} \left(R \bowtie_{\text{R.AID = Author.AID}} \text{Author} \right)$$

(XII) Retrieve ISBN and average rating given to each book

$$\gamma_{\text{ISBN, avg(Rating)}} (\text{Review})$$

(XIII) Retrieve the name of the publishers who have published all categories of books.

$$\text{Result} \leftarrow \Pi_{\text{pid, pname}}^{(\text{Publishers})} \div \Pi_{\text{pid}}^{(\text{Books})}$$

$$\text{result} \leftarrow \Pi_{\text{pname}} (R \bowtie_{\text{R.pid = Publishers.pid}} \text{Publishers})$$

$$\Pi_{\text{pname, category}} (\text{Books} \bowtie_{\text{Books.pid = Publishers.pid}} \text{Publishers}) \div \Pi_{\text{category}}^{(\text{Books})}$$

Relational Calculus

(i) Tuple Relational Calculus
(ii) Domain Relational Calculus

Tuple

$\{T \mid P(T)\}$ → all tuples in table 'P'

- To get name of employee whose sal > 10000

$\{t.name \mid emp(t) \wedge t.sal > 10000\}$

- if F_1 and F_2 are two formulas, we can have:

$F_1 \wedge F_2$, $F_1 \vee F_2$, $F_1 \Rightarrow F_2$, $\neg F_1$, $\neg F_2$

Quantifier
- Universal ($\forall$) – "for all"
- Existential ($\exists$) – "there exists"

$F_1 \wedge F_2$ → true if both are true
$F_1 \vee F_2$ → false if both are false
$F_1 \Rightarrow F_2$ → false if F_1 is true but F_2 is false

$(\exists T)(F)$ → at least one tuple assigned to free occurences of T in F_1 then true, otherwise false
$(\forall T)(F)$ → if F is true for every tuple assigned to free occurences of T in F_1 then true, else false.

Domain

Author (a_0 , a_1 , a_2 , a_3 , a_4 , a_5 , a_6)
(aid, aname, state, city, zip, phone, url)

- city, phone, url where aname = 'Ram'

$\{a_3, a_5, a_6 \mid (\exists a_1)\ Author(a_0, a_1, a_2, a_3, a_4, a_5, a_6) \wedge a_1 = 'Ram'\}$

- name, address, phone of publishers in state='Bihar'

$\{P_1, P_2, P_4 \mid (\exists P_3)\ Publisher(P_0, P_1, P_2, P_3, P_4, P_5) \wedge P_3 = 'Bihar'\}$

- id, name, address, phone of publishers publishing novels

$\{P_0, P_1, P_2, P_4 \mid (\exists b_2)(\exists b_3)\ Book(b_0, b_1, b_2, b_3, b_4, b_5, b_6, b_7, b_8) \wedge Publisher(P_0, P_1, P_2, P_3, P_4, P_5)$
$\wedge b_4 = P_0 \wedge b_5 = 'Novel'\}$